

SERVICENOW

Scripting Mastery

A Beginner-to-Advanced Guide to Business Rules, Client Scripts,
Script Includes, GlideAjax, and Platform Automation

Compiled by V. Mahamadh Basha | ServiceNow CSA & CAD Certified

Sourced from official ServiceNow product documentation and developer resources

Edition: 2026

```
}  
gs.info(gr.getValue('number'));  
while (gr.next()) {  
  gr.query();  
  gr.addQuery('active', true);  
  var gr = new GlideRecord('incident');
```

Table of Contents

Part I — Foundations (Beginner)

1. What Is ServiceNow Scripting?	3
2. The Scripting Environment & JavaScript Engine	3
3. Core Data Object: GlideRecord	4
4. The Global Scripting Object: GlideSystem (gs)	5
5. Business Rules — Automating the Server Side	6
6. Client Scripts — Reacting to User Input	7

Part II — Building Proficiency (Intermediate)

7. UI Policies vs. Client Scripts	8
8. Script Includes — Reusable Server Logic	9
9. GlideAjax — Client-to-Server Communication	10
10. The GlideForm API (g_form)	11
11. Access Control Rules (ACL) Scripting	11
12. Scoped Applications & the Now Platform API	12

Part III — Advanced Engineering

13. Flow Designer & IntegrationHub Scripting	13
14. Scripted REST APIs & Outbound Integrations	14
15. Performance, Governance & Secure Coding	15
16. Debugging, Logging & the Automated Test Framework	16
17. AI-Augmented Development: Now Assist & 2026 Trends	17

Part IV — Wrap-Up

18. Summary & Key Takeaways	18
19. Further Reading & Official Resources	18

How to Use This Guide

This guide is organized into three progressive parts. Part I assumes no prior ServiceNow development experience and introduces the platform's scripting model from the ground up. Part II builds fluency with the tools used in day-to-day development. Part III covers advanced engineering practices used by ServiceNow Developers, CIS specialists, and Platform Architects. Each chapter includes runnable code examples, a hands-on exercise, and a callout box highlighting a common pitfall or best practice.

All technical references are grounded in official ServiceNow documentation (docs.servicenow.com and developer.servicenow.com) and the ServiceNow Developer Program. Where product terminology has evolved across releases (for example, the renaming of the platform to the 'Now Platform' / 'ServiceNow AI Platform'), the most current official term is used.

PREREQUISITE

A free Personal Developer Instance (PDI) from developer.servicenow.com is recommended so you can follow along with every exercise in a real environment.

PART I

Foundations

Beginner Level — building your mental model of the platform

1. What Is ServiceNow Scripting?

ServiceNow scripting refers to the JavaScript-based customization layer built into the Now Platform that lets administrators and developers extend out-of-the-box behavior: validating data, automating approvals, integrating with external systems, and building custom applications. According to ServiceNow's own platform documentation, the underlying scripting engine executes server-side JavaScript using the Rhino engine on older releases and has since migrated to GraalJS on current releases, while client-side scripts run standard browser JavaScript.

Scripting happens in five main contexts:

- **Server-side** — Business Rules, Script Includes, Scheduled Jobs, Scripted REST APIs.
- **Client-side** — Client Scripts, UI Policies (scripted), Catalog Client Scripts.
- **Workflow / Flow** — Script steps inside Flow Designer and legacy Workflow Editor.
- **Data layer** — Background Scripts (Scripts – Background) for one-off admin tasks.
- **Integration layer** — Scripted REST/SOAP APIs, MID Server scripts, Transform Map scripts.

WHY IT MATTERS FOR THE BUSINESS

Every hour of well-designed scripting removes repetitive manual work from HR, IT, and customer service teams. Recruiters and hiring managers should treat 'scripting fluency' as a strong signal of a candidate's ability to translate business rules into automated outcomes.

2. The Scripting Environment & JavaScript Engine

All scripts are written and tested inside Studio (the primary IDE for scoped application development) or directly on individual configuration records such as the Business Rule or Client Script form. ServiceNow's developer documentation recommends always developing inside a scoped application rather than the Global scope, since scoped apps enforce namespace isolation, controlled API access, and safer upgrades.

EXAMPLE — BACKGROUND SCRIPT

```
// A minimal server-side script executed from
// Background Scripts (System Definition > Scripts - Background)
var gr = new GlideRecord('incident');
gr.addQuery('active', true);
gr.query();
gs.info('Active incidents: ' + gr.getRowCount());
```

EXERCISE 1

Task: Open your Personal Developer Instance, navigate to Scripts - Background, and run a script that counts how many open Incidents are assigned to your user.

Hint: Use GlideRecord on the 'incident' table with an addQuery on 'assigned_to' and 'active'.

3. Core Data Object: GlideRecord

GlideRecord is the primary server-side (and a limited client-side) API used to query, insert, update, and delete platform records. ServiceNow's API reference documents it as a class of object that represents a database table and exposes a scriptable interface to interact with records on that table without writing SQL.

EXAMPLE — QUERYING & UPDATING RECORDS

```
// Query and update pattern (server-side Business Rule / Script Include)
var gr = new GlideRecord('incident');
gr.addQuery('priority', 1);
gr.addQuery('active', true);
gr.query();

while (gr.next()) {
    gr.work_notes = 'Escalated automatically by SLA policy';
    gr.assignment_group = '8a4dde73c6112278017a6a4baf547aa7'; // sys_id
    gr.update();
}
```

Key GlideRecord methods every beginner must know:

- **addQuery(field, value)** — adds a filter condition (AND by default).
- **query()** — executes the query against the database.
- **next()** — advances the record pointer; returns false when no records remain.
- **get(sys_id)** — retrieves a single record directly by unique identifier.
- **insert() / update() / deleteRecord()** — persist changes to the table.
- **getValue(field) / setValue(field, value)** — read/write the underlying (unformatted) value.

COMMON PITFALL

Never call `.query()` inside a loop that also iterates another `GlideRecord` — this is the single most common cause of severe performance issues in ServiceNow instances. Use `GlideAggregate` or a single joined query instead.

4. The Global Scripting Object: GlideSystem (gs)

GlideSystem, referenced through the global 'gs' object, provides system-level utility methods available in every server-side script: logging, session information, date/time handling, and messages shown to users.

EXAMPLE — GLIDESYSTEM ESSENTIALS

```
gs.info('Diagnostic message written to the system log');
gs.addErrorMessage('Please provide a valid cost center before submitting.');
```

```
var today = gs.nowDateTime();
var currentUser = gs.getUserDisplayName();
```

EXERCISE 2

Task: Write a Business Rule that adds an informational message to the user (gs.addInfoMessage) when a Change Request is set to state 'Implement' without an assigned CAB approval.

Hint: Combine a 'before' Business Rule with gs.addInfoMessage() and a conditional on current.state.

5. Business Rules — Automating the Server Side

Business Rules are the most common server-side automation tool in ServiceNow. They run when a record is displayed, inserted, updated, or deleted, and can execute before or after the database operation, or asynchronously for non-blocking, longer-running logic.

The four Business Rule execution types:

- **Before** — runs before the record is saved; ideal for validation and field defaulting.
- **After** — runs after the record is saved; ideal for downstream updates to related records.
- **Async** — runs in a scheduled background job; used for long-running or non-urgent logic.
- **Display** — runs when the form loads, passing data to the client via g_scratchpad.

EXAMPLE — BEFORE BUSINESS RULE

```
// Before Business Rule on Incident: auto-set impact from category
(function executeRule(current, previous /*null when async*/) {
  if (current.category == 'network' && current.priority == 1) {
    current.impact = 1;
  }
})(current, previous);
```

HR / BUSINESS RELEVANCE

In HR Service Delivery, Business Rules commonly auto-route employee cases to the correct HR Service department based on the case topic, cutting manual triage time significantly.

EXERCISE 3

Task: Create a Business Rule on the HR Case table that automatically sets 'priority' to High whenever the case topic equals 'Payroll' and due date is within 24 hours.

Hint: Use a 'before update or insert' rule with a GlideDateTime comparison on due date.

6. Client Scripts — Reacting to User Input

Client Scripts run in the user's browser and manipulate the form in real time using the GlideForm API (g_form). They come in three types: onLoad, onChange, and onSubmit.

EXAMPLE — ONCHANGE CLIENT SCRIPT

```
function onChange(control, oldValue, newValue, isLoading) {
  if (isLoading || newValue === '') return;

  if (newValue == 'network') {
    g_form.setMandatory('assignment_group', true);
    g_form.showFieldMsg('assignment_group',
      'Assignment group is required for Network incidents', 'info');
  } else {
    g_form.setMandatory('assignment_group', false);
  }
}
```

COMMON PITFALL

Client Scripts should never contain direct GlideRecord calls against large tables — that logic belongs on the server via GlideAjax (covered in Part II) to avoid exposing business logic and slowing the browser.

PART II

Building Proficiency

Intermediate Level — the tools used in daily development

7. UI Policies vs. Client Scripts

UI Policies provide a declarative (mostly no-code) way to show/hide, require, or read-only fields based on conditions, and ServiceNow's own scripting best-practice guidance recommends using UI Policies over Client Scripts whenever the requirement is purely visual, since they are easier to maintain and carry lower performance overhead.

Use a Client Script only when you need:

- Custom validation logic beyond simple field comparisons.
- Dynamic messages, popups, or calling GlideAjax.
- Cross-field calculations that a UI Policy cannot express declaratively.

8. Script Includes — Reusable Server Logic

A Script Include defines a reusable, on-demand JavaScript class that other server-side scripts can call. Because Script Includes only load into memory when invoked, ServiceNow documents them as the preferred way to encapsulate reusable logic without impacting unrelated transactions.

EXAMPLE — SCRIPT INCLUDE CLASS

```
var IncidentUtils = Class.create();
IncidentUtils.prototype = {
    initialize: function() {},

    getOpenCountForGroup: function(groupSysId) {
        var ga = new GlideAggregate('incident');
        ga.addQuery('assignment_group', groupSysId);
        ga.addQuery('active', true);
        ga.addAggregate('COUNT');
        ga.query();
        if (ga.next()) {
            return ga.getAggregate('COUNT');
        }
        return 0;
    },

    type: 'IncidentUtils'
};
```

EXERCISE 4

Task: Refactor Exercise 1's Background Script logic into a Script Include method called 'getMyOpenIncidentCount' and call it from a Business Rule.

Hint: Remember Script Includes must set 'type' equal to the class name and be marked Client callable only if invoked from GlideAjax.

9. GlideAjax — Client-to-Server Communication

GlideAjax lets a Client Script call a server-side Script Include asynchronously without a full page submit — the standard, supported pattern for retrieving server data or running server logic from the browser.

EXAMPLE — GLIDEAJAX CALL

```
// Client Script
var ga = new GlideAjax('IncidentUtils');
ga.addParam('sysparm_name', 'getOpenCountForGroup');
ga.addParam('sysparm_group', g_form.getValue('assignment_group'));
ga.getXMLAnswer(function(answer) {
    g_form.showFieldMsg('assignment_group',
        'Open incidents in this group: ' + answer, 'info');
});
```

SECURITY NOTE

Script Includes called via GlideAjax must set 'Client callable' to true and must never trust client-supplied parameters blindly — always re-validate on the server.

10. The GlideForm API (g_form)

g_form is the client-side API for reading and manipulating form state: field values, visibility, mandatory status, and UI messages.

- **getValue(field) / setValue(field, value)** — read/write field data.
- **setMandatory(field, boolean)** — toggle required status.
- **setVisible(field, boolean) / setReadOnly(field, boolean)** — control field display.
- **addInfoMessage() / addErrorMessage()** — surface feedback to the user.

11. Access Control Rules (ACL) Scripting

ACLs can include a scripted condition evaluated in addition to role checks, returning true or false to grant or deny access at the record or field level — the platform's core security enforcement mechanism as documented in ServiceNow's security best-practice guidance.

EXAMPLE — ACL SCRIPT

```
// Scripted condition on a 'write' ACL for the HR Case table
answer = gs.getUser().isMemberOf('HR Case Manager') ||
    current.opened_by == gs.getUserID();
```

12. Scoped Applications & the Now Platform API

Scoped applications isolate custom code and data behind a defined application scope, restricting cross-scope access unless explicitly permitted via Scoped GlideRecord or Cross-Scope Access Rules — ServiceNow's recommended architecture for all new development on the Now Platform.

EXERCISE 5

Task: Create a small scoped application called 'x_yourscope_hrtools' containing one Script Include and one Business Rule, then attempt to access it from Global scope to observe the Cross-Scope Access Rule prompt.

Hint: Studio > Create Application, then check System Applications > Cross Scope Access after.

PART III

Advanced Engineering

Advanced Level — architecture, integration, and governance

13. Flow Designer & IntegrationHub Scripting

Flow Designer is ServiceNow's low-code automation engine, but complex logic still requires a Script step or a custom Action using server-side JavaScript, particularly for data transformations that declarative actions cannot express.

EXAMPLE — FLOW DESIGNER SCRIPT STEP

```
// Flow Designer "Script" step
(function execute(inputs, outputs) {
    var gr = new GlideRecord('sc_req_item');
    gr.addQuery('sys_id', inputs.request_item_id);
    gr.query();
    if (gr.next()) {
        outputs.days_open = gs.dateDiff(gr.getValue('sys_created_on'),
                                        gs.nowDateTime(), true);
    }
})(inputs, outputs);
```

14. Scripted REST APIs & Outbound Integrations

Scripted REST APIs let developers expose custom endpoints, while GlideHTTPRequest / RESTMessageV2 handle outbound calls to external systems — the two building blocks behind most ServiceNow-to-third-party integrations.

EXAMPLE — SCRIPTED REST API

```
// Scripted REST Resource - GET
(function process(request, response) {
  var gr = new GlideRecord('incident');
  gr.addQuery('sys_id', request.pathParams.incident_id);
  gr.query();
  if (gr.next()) {
    response.setBody({
      number: gr.getValue('number'),
      state: gr.getDisplayValue('state')
    });
  } else {
    response.setStatus(404);
  }
})(request, response);
```

15. Performance, Governance & Secure Coding

ServiceNow's platform performance documentation and Well-Architected Framework guidance converge on the same core rules for production-grade scripts:

- Never nest GlideRecord queries inside loops — use GlideAggregate or dot-walking instead.
- Always set `gr.setLimit()` when you only need a bounded result set.
- Prefer Script Includes (lazy-loaded) over Business Rule inline logic for shared code.
- Avoid synchronous business logic on high-volume tables; use Async Business Rules or Flow.
- Sanitize and validate all client-supplied input before using it in a query or update.
- Use the Now Platform's built-in ATF and Code Search before deploying to production.

GOVERNANCE TIP

Enable and review the 'Business Rule / Script performance' entries in the System Diagnostics > Slow Query Log regularly — this is the same technique ServiceNow's own performance consultants use to triage instance slowness.

16. Debugging, Logging & the Automated Test Framework

Effective debugging combines `gs.info()`/`gs.error()` logging, the in-platform Script Debugger, and the Automated Test Framework (ATF) for regression-proofing scripted behavior across upgrades.

EXAMPLE — DEFENSIVE ERROR HANDLING

```
try {
  var gr = new GlideRecord('incident');
  gr.get(current.getValue('parent_incident'));
  gr.child_incidents += 1;
  gr.update();
} catch (e) {
  gs.error('IncidentUtils: failed to update parent incident - ' + e.message);
}
```

EXERCISE 6

Task: Build an ATF test that creates an Incident with category 'network' and priority 1, then asserts that impact is automatically set to 1 by the Business Rule from Exercise 3.

Hint: Use the 'Create Record', 'Impersonate User', and 'Field Value Assertion' ATF test steps.

17. AI-Augmented Development: Now Assist & 2026 Trends

ServiceNow's 2026 platform direction, described in its workflow automation outlook with Deloitte, centers on AI-ready architecture and process transformation with AI embedded at the core of enterprise workflows, including AI agents that can act on structured business logic. In practical terms for developers, this means:

- **Now Assist for Development** — AI-generated Script Include and Business Rule scaffolding that developers review and refine rather than write from scratch.
- **Agentic AI workflows** — AI agents that call the same scripted APIs (Script Includes, Scripted REST APIs) developers already build, making clean, well-documented server-side APIs more valuable than ever.
- **Governance-first AI** — ServiceNow leadership has emphasized that trust and governance frameworks, not just automation speed, will define which organizations succeed with AI in 2026.

CAREER INSIGHT

Industry career guidance for 2026 consistently lists scripting, API integration, and data import proficiency as the core technical requirements for ServiceNow Developer and Application Developer roles — making this guide's Part I and II material the highest-leverage skill set for fresher and early-career candidates.

18. Summary & Key Takeaways

ServiceNow scripting spans a spectrum from declarative, low-code configuration to full server-side and client-side JavaScript engineering. Mastery follows a natural progression: understand GlideRecord and GlideSystem first, then automate with Business Rules and Client Scripts, then modularize logic with Script Includes and GlideAjax, and finally scale that knowledge into scoped applications, integrations, and AI-augmented workflows.

Key Takeaways

1. GlideRecord and GlideSystem (gs) are the two foundational APIs behind almost every server-side script.
2. Business Rules automate server-side logic on record operations (before, after, async, display).
3. Client Scripts and UI Policies handle real-time form behavior — prefer UI Policies when the need is purely visual.
4. Script Includes plus GlideAjax are the supported pattern for reusable logic and client-server communication.
5. ACL scripts enforce security at the record and field level and should always be least-privilege by default.
6. Scoped applications are the standard architecture for new development, isolating code and enforcing controlled cross-scope access.
7. Performance discipline — avoiding nested queries, using GlideAggregate, and async processing — separates production-ready scripts from fragile ones.
8. ATF testing and disciplined logging (gs.info/gs.error) are essential for safe upgrades and maintainable code.
9. AI-assisted development (Now Assist) is accelerating scaffolding, but clean, well-documented APIs remain the developer's core value-add in 2026.

19. Further Reading & Official Resources

ServiceNow Product Documentation

<https://www.servicenow.com/docs/> — GlideRecord, GlideSystem, GlideAjax, and full server/client API reference.

ServiceNow Developer Portal

<https://developer.servicenow.com/> — free Personal Developer Instance, API references, and learning plans.

ServiceNow Developer Blog & Community

<https://www.servicenow.com/community/> — API discussions, code samples, and MVP-authored best practices.

ServiceNow Now Learning

<https://nowlearning.servicenow.com/> — official CSA, CAD, and CIS certification learning paths.

Automated Test Framework Documentation

Search 'Automated Test Framework' on docs.servicenow.com for ATF step reference and best practices.

2026 Workflow Automation Outlook (Deloitte x ServiceNow)

<https://www.deloitte.com/global/en/alliances/servicenow/about/2026-workflow-automation-outlook.html> — AI and

workflow trends shaping platform direction.

This guide is an independent educational resource compiled from official ServiceNow documentation and public platform announcements. It is not published by or affiliated with ServiceNow, Inc. All product names and trademarks belong to their respective owners.